# 数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 11, 2026

# Contents


1. 当日起点：项目状态与工具状态 ..... 4
   1.1. 进度概览 ..... 4
   1.2. 待办 ..... 4
2. 有什么方法可以方便查阅 Go 的关键字 ..... 5
   2.1. 费曼直觉 ..... 5
   2.2. 硬核机制 ..... 5
      2.2.1. 25 个关键字（本机 Go 1.27.1 现抽，按用途分五组） ..... 5
      2.2.2. 三条离线路径 + 一条在线路径 ..... 5
      2.2.3. 为什么 `go doc` 查不到关键字 ..... 6
      2.2.4. 编译器眼里的关键字：`go/token` ..... 6
   2.3. 陷阱与易错 ..... 7
   2.4. 关键点 ..... 8
   2.5. 变式检验 ..... 9
   2.6. 同类归类 ..... 9
   2.7. Go 语言机制补充 ..... 9
   2.8. 本次操作与当前状态 ..... 10
3. 学习实验用的 scratch/ 目录怎么设计 ..... 11
   3.1. 费曼直觉 ..... 11
   3.2. 硬核机制 ..... 11
      3.2.1. 落地的结构 ..... 11
      3.2.2. 三条硬规则（都是工具链定的，不是习惯） ..... 11
      3.2.3. 为什么实验区必须待在模块内 ..... 12
   3.3. 陷阱与易错 ..... 12
   3.4. 关键点 ..... 13
   3.5. 变式检验 ..... 13
   3.6. 同类归类 ..... 13
   3.7. Go 语言机制补充 ..... 13
   3.8. 本次操作与当前状态 ..... 14
4. gokw 怎么用 ..... 15
   4.1. 费曼直觉 ..... 15
   4.2. 硬核机制 ..... 15
      4.2.1. 四条命令 ..... 15
      4.2.2. `gokw <词>` 的四种结论 ..... 15
      4.2.3. 为什么它不会过期 ..... 15
      4.2.4. fish 是怎么找到它的 ..... 16
   4.3. 陷阱与易错 ..... 16
   4.4. 关键点 ..... 16
   4.5. 变式检验 ..... 17
   4.6. 同类归类 ..... 17
   4.7. Go 语言机制补充 ..... 17
   4.8. 本次操作与当前状态 ..... 18

Index of Listings ..... 19

# 1. 当日起点：项目状态与工具状态

这一节记录 10-11 开工时的状态（距上次记录 10-07 已过 4 天），作为今天的基线。

## 1.1. 进度概览

| 项 | 状态 |
| --- | --- |
| `internal/db/kv.go` | `Del` 仍只有签名 → `go build ./...` 停在 `internal/db/kv.go:28:6: missing function body` |
| `Get` 的语义 | **冲突仍在**：实现里未命中的分支返回 `errors.New("the key is not existed!")`，而 10-09 重写过的 `kv_test.go` 第 19、22 行要求 `!ok && err == nil` |
| 测试位置 | **已迁移**：10-09 把 `test/` 下的黑盒测试整体挪进 `internal/db/`，改成与实现同包的 `package db`（白盒）；`test/` 目录已不存在 |
| `go test ./...` | ✗ `build failed`（同一个 `missing function body`） |
| `gofmt` | 未跑 |

> ⚠️ **陷阱预警** 10-09 那次迁移**不是简单搬家**：`kv_test.go` 的期望被改写成「未命中 key 不算错误」。所以补 `Del` 只换来“能编译”，`Get` 这一条不动，测试照样红。

## 1.2. 待办

1. **补 `Del`**：`delete` 对不存在的键是安全空操作、不会告诉你删没删到，所以要先查一次拿到 `deleted`；
2. **定 `Get` 未命中的语义**：按新测试应当是 `return nil, false, nil`（10-07 那节记的正是这条）；
3. **可选**：`Set` 的 `updated` 按课程语义写（现在写死的 `true`）；
4. 收尾四步：`gofmt -l .` → `go build ./...` → `go vet ./...` → `go test ./...`。

`AGENTS.md` 里「`test/` 是黑盒测试目录」那段，今天已按现状改成「测试在 `internal/db/`、白盒 `package db`」。

# 2. 有什么方法可以方便查阅 Go 的关键字

## 2.1. 费曼直觉

关键字（keyword）不是“标准库里的东西”。标准库是你 `import` 进来的一堆代码，而关键字是**语言的骨架词**：编译器读到源码第一个字节时（词法扫描阶段）就把它们认出来了，语法解析器也只为这些词准备了语法规则。所以查它们的正确去处，和查 API 完全不同：

| 想查的东西 | 去哪儿 | 为什么 |
| --- | --- | --- |
| 关键字（`func`、`range`、`defer`） | **语言规范** + 编译器源码 | 它是语法的一部分，不在任何包里 |
| 预声明标识符（`len`、`nil`、`any`） | `go doc builtin` | 它们在 `builtin` 这个伪包里真有条目 |
| 标准库 API（`slices.Delete`） | `go doc <包>.<符号>` | 那才是“文档”，随包一起发布 |

一句话：**关键字查规范，内建名查 builtin，API 查 go doc。**

## 2.2. 硬核机制

### 2.2.1. 25 个关键字（本机 Go 1.27.1 现抽，按用途分五组）

| 分组（个数） | 关键字 | 一句话 |
| --- | --- | --- |
| 包与声明（6） | `package` `import` `const` `var` `type` `func` | 声明包、引入包，声明常量/变量/类型/函数 |
| 流程控制（11） | `if` `else` `for` `range` `switch` `case` `default` `break` `continue` `goto` `fallthrough` | 分支与循环的全部语法 |
| 返回与收尾（2） | `return` `defer` | 函数出口；`defer` 是“推迟到返回前执行” |
| 并发（3） | `go` `chan` `select` | 起协程、通道、多路等待 |
| 类型与结构（3） | `struct` `interface` `map` | 三种复合类型字面量的开头 |

**总计 6 + 11 + 2 + 3 + 3 = 25。**

### 2.2.2. 三条离线路径 + 一条在线路径

| 路径 | 怎么用 | 特点 |
| --- | --- | --- |
| `gokw`（今天装的） | `gokw`、`gokw range`、`gokw spec` | 一屏看完，含中文说明 |
| 离线语言规范 | `/usr/lib/go/doc/go_spec.html` 第 255-266 行 | 最权威，有英文原文 |
| 编译器源码 | `/usr/lib/go/src/go/token/token.go` 第 95-126 行 | 看清“谁在管这些词” |
| 在线规范 | `https://go.dev/ref/spec#Keywords` | 给人发链接时用 |

四条路径的内容必须一致 —— 不一致就是 Go 版本或文件被改过了。

### 2.2.3. 为什么 `go doc` 查不到关键字

`go doc` 的参数被解释为**包路径或包里的符号**（`go help doc` 原文：If there is no package specified or matched, the package in the current directory is selected）。于是在项目里敲：

```
$ go doc range
doc: no symbol range in package github.com/zz-cpp/db-go/internal/db
```

Listing 1: 它把 range 当成“当前包里一个叫 range 的符号”，找不到就报这个 —— 而不是告诉你「这是关键字」

`go doc builtin` 里也没有它：关键字不在任何包里。

### 2.2.4. 编译器眼里的关键字：`go/token`

关键字被建模成**词法单元**（token），和 `+`、`(`、`;` 是同一类东西，只是值域不同：

```
// /usr/lib/go/src/go/token/token.go:95-126（节选）
  keyword_beg
  // Keywords
  BREAK
  CASE
  CHAN
  CONST
  CONTINUE
  // 中间 20 个同形，此处从略
  VAR
  keyword_end
```

Listing 2: `keyword_beg` / `keyword_end` 是一对哨兵：夹在中间的就是关键字

> 📌 **核心要点** `keyword_beg … keyword_end` 这对哨兵让“是不是关键字”退化成一次**整数区间比较**：`Token.IsKeyword()` 的实现就是 `keyword_beg < tok && tok < keyword_end`（token.go:313），O(1) 且不查表。beg/end 不是为了好看，是为了让判定不必遍历。

词法扫描器怎么用它 —— 初始化时把区间里每个词塞进 map：

```go
// /usr/lib/go/src/go/token/token.go:282-297
var keywords map[string]Token

func init() {
  keywords = make(map[string]Token, keyword_end-(keyword_beg+1))
  for i := keyword_beg + 1; i < keyword_end; i++ {
    keywords[tokens[i]] = i
  }
}

// Lookup maps an identifier to its keyword token or [IDENT] (if not a keyword).
func Lookup(ident string) Token {
  if tok, is_keyword := keywords[ident]; is_keyword {
    return tok
  }
  return IDENT
}
```

Listing 3: 一次哈希查询就把“标识符还是关键字”分好类；没命中就是普通标识符 IDENT

之后 scanner 在每个“字母开头的字面量”上调用它：

```go
// /usr/lib/go/src/go/scanner/scanner.go:842-851 (节选)
  switch ch := s.ch; {
  case isLetter(ch):
    lit = s.scanIdentifier()
    if len(lit) > 1 {
      // keywords are longer than one letter - avoid lookup otherwise
      tok = token.Lookup(lit)
      // 随后按 token 决定是否补分号 (IDENT/BREAK/CONTINUE/FALLTHROUGH/RETURN 要补)
    }
```

Listing 4: 单字母标识符（i、x）不可能撞上关键字，所以干脆跳过查询 —— 热路径上的小优化

> 💡 **背景与类比** `go/token` + `go/scanner` + `go/parser` 是标准库里**给工具用的**前端三件套：gofmt、go vet、gopls 都靠它。编译器自己（`/usr/lib/go/src/cmd/compile`）另有实现，但关键字清单同出一份语言规范。

## 2.3. 陷阱与易错

**陷阱 1：关键字 ≠ 预声明标识符。** `len`、`cap`、`nil`、`true`、`any`、`min`、`max`、`clear` 都只是“预先声明在你作用域里的名字”，可以合法地遮蔽（shadow）它们：

```go
package main

import "fmt"

func main() {
  len := 42 // 合法：这个 len 只是遮蔽了内建函数
  fmt.Println(len)
}
```

Listing 5: 实测输出 42，编译通过 —— 换成任何关键字都不行

**陷阱 2：用关键字当名字，报的是语法错而不是词法错。** 扫描器**认得**这些词（它们被正常扫描成关键字 token），是解析器在“期待标识符”的位置撞见它们才拒绝：

```
$ go run ./func_as_name
func_as_name/main.go:4:7: syntax error: unexpected :=, expected (

$ go run ./if_as_name
if_as_name/main.go:4:5: syntax error: unexpected :=, expected expression
if_as_name/main.go:5:6: syntax error: unexpected keyword if, expected {
if_as_name/main.go:6:1: syntax error: unexpected }, expected expression
```

Listing 6: 注意第二段里那句 unexpected keyword if —— 编译器明确告诉你它认出了关键字

**陷阱 3：`any`、`comparable`、`min`、`max`、`clear` 都不是关键字。** Go 1.18 / 1.21 引入它们时特意做成预声明标识符，老代码里叫 `min` 的变量升级后不会突然编译不过。这条可以推导：**只要能做成内建名，Go 就不动关键字表。**

**陷阱 4：`go doc` 查关键字只会给出误导性的报错**（见上一节），别把它当成“没有这个关键字”。

**陷阱 5：写脚本时的两个环境坑**（今天都踩到了）。① 本机 fish 里 `grep` 是 `function grep --wraps=rg` 的包装，`grep -E` 会被 rg 当成 `--encoding` 报 `unknown encoding` —— 脚本里要用 `command grep`；② `go doc builtin` 的输出里 `func` 条目带 4 个空格缩进（`    func len(v Type) int`），只有 `type` 顶格，正则漏掉 `^[[:space:]]*` 就会漏判 `len`、`nil`、`append`。

> ⚠️ **陷阱预警** 别用“关键字清单”去判断一个词能不能当标识符 —— 那条规则是**两段式**的：先看是不是关键字（保留字，任何作用域都禁止），再看是不是预声明标识符（只是作用域里预置的名字，允许遮蔽）。`token.IsIdentifier()` 的源码把这点写死了：`if name == "" || IsKeyword(name) { return false }`。

## 2.4. 关键点

1. 关键字共 **25 个**，五组：包与声明 6、流程控制 11、返回与收尾 2、并发 3、类型与结构 3。
2. “查关键字”= 查语言规范，不是查文档：离线规范 `/usr/lib/go/doc/go_spec.html` 第 255-266 行，在线 `https://go.dev/ref/spec#Keywords`。

3. 机器可读的权威来源是编译器自己的 `go/token` 包：`keyword_beg` … `keyword_end` 区间 + `keywords` map + `Lookup` 。
4. 日常速查用今天装的 `gokw`：清单每次从 GOROOT 源码现抽，Go 升级后不会过期。
5. 关键字是保留字（**不可**遮蔽），内建名不是（**可**遮蔽）—— 新手最容易混的一条。

## 2.5. 变式检验

1. 我想用 `len` 当变量名，行吗？→ 行，内建名可遮蔽（实测输出 42）。那 `map` 呢？→ 不行，关键字，`map := 1` 直接语法错。
2. `go doc range` 报 `no symbol range in package …`，是说明这个库里没有 range 吗？→ 不是。`go doc` 把参数当“当前包的符号名”找；查关键字要去规范或 `go/token` 。
3. `if := 3` 的错误为什么算“语法错误”而不是“词法错误”？ → 扫描器正常产出了关键字 token，是解析器在“期待标识符”的位置撞见它才报错 —— 错误文本里直接写着 `unexpected keyword if` 。
4. 下个 Go 版本要加新语法，会往清单里加关键字吗？→ 能不做就不做：`any` 、`comparable` 、`min` 、`max` 、`clear` 全部走预声明标识符路线，就是为了不动关键字表。
5. 我把 `gokw` 删了，还能查吗？→ 能，三条路都在：`gokw spec` 对应的规范文件、`go/token` 源码、在线规范；最坏情况用一条 awk 现抽（见下节）。

## 2.6. 同类归类

Go 里的“名字”分四层，查法各不相同：

| 层 | 例子 | 去哪查 / 能否遮蔽 |
|---|---|---|
| 关键字 | `func` `range` `defer` | 语言规范；**不可**遮蔽 |
| 预声明标识符 | `len` `nil` `any` `true` `iota` | `go doc builtin`；可遮蔽 |
| 标准库导出名 | `slices.Delete` `fmt.Println` | `go doc <包>.<符号>`；可同名遮蔽 |
| 你自己的名字 | `KV` `kv.mem` | `go doc ./internal/db.KV` （相对路径） |

## 2.7. Go 语言机制补充

**`type Token int` 而不是字符串枚举。** `go/token` 里所有词法单元都是 `int` 常量，按“字面量 → 运算符 → 关键字 → 附加”分段，每段两端各有一个 `xxx_beg` / `xxx_end` 哨兵。这样判定一律是整数比较；要把 token 变回文字，用的是 `token.go:203` 起的一张 `tokens` 表（下标是 token、值是它的拼写）。你在报错里看到的 `unexpected keyword if` ，就是这张表查出来的。

**“给工具用”的前端与编译器前端是两套实现。**标准库的 `go/scanner` 、`go/parser` 、`go/token` 服务于 gofmt / vet / gopls；`cmd/compile` 内部有自己的扫描器。两者关键字清单同出一份语言规范，所以不会打架。

**可推导 vs 只能记。** `keyword_beg < tok && tok < keyword_end` 这类哨兵区间用法能从源码推出来；而“关键字恰好是这 25 个、且多年未变”是**规范决定的事实**，推不出来，只能查 —— 记忆锚点：`any` 不是关键字，`defer` 是。

**验证命令**（可直接复制）：

```
$ gokw                       # 25 个关键字分组表
$ gokw len                   # 判定单个词（关键字 / builtin / 普通标识符）
$ gokw spec                  # 离线规范 #Keywords 原文
$ go doc go/token.IsKeyword # 机制：IsKeyword 的官方文档
$ awk '/^\tkeyword_beg/{f=1; next} /^\tkeyword_end/{f=0} \
      f && $1 ~ /^[A-Z]+$/ {print tolower($1)}' \
      "$(go env GOROOT)/src/go/token/token.go"
```

Listing 7: 最后一条就是 gokw 内部用的抽取命令，可以直接粘来核对当前 Go 版本

## 2.8. 本次操作与当前状态

### 新增的文件

1. `note/2026-10-11.typ`（本文件；头部逐字节复制自 `2026-10-07.typ`，`diff` 已验证一致）及其同名 PDF；
2. `~/.config/fish/functions/gokw.fish` —— 新装的 fish 函数，支持 `gokw` / `gokw <词>` / `gokw spec` / `gokw -h`。它没有改动任何既有文件（`godocbat` 家族原样未动）；不要了就 `rm ~/.config/fish/functions/gokw.fish`。

### 跑过的验证

```
$ fish -c 'gokw'          # 25 个关键字，分组 + 中文说明 ✓
$ fish -c 'gokw range'    # 关键字 ✓
$ fish -c 'gokw len'      # 预声明标识符 ✓（修掉 go doc builtin 缩进 bug 后）
$ fish -c 'gokw myvar'    # 普通标识符 ✓
$ fish -c 'gokw spec'     # 离线规范 255-266 行原文 ✓
$ fish -c 'gokw -h'       # 用法 ✓（修掉 fish 把 -h 当选项的坑后）
$ go run ./len_shadow     # 42 —— 内建名可被遮蔽 ✓
$ go run ./func_as_name   # syntax error: unexpected :=, expected ( ✓
```

Listing 8: gokw 子命令与两个反例程序均实测（反例程序放在 /tmp/kwtest，未进仓库）；PDF 由 typst compile 零错误生成

### 今天动过的其它文件

1. `AGENTS.md`：「`test/` 是黑盒测试目录」一段已按现状改成「测试在 `internal/db/`、白盒 `package db`」。

**项目本身没动**：本次是“查资料 + 装工具”。`internal/db/kv.go` 仍是 `Del` 缺函数体、`go build ./...` 仍报 `missing function body` —— 待办见上面的「当日起点」。

# 3. 学习实验用的 scratch/ 目录怎么设计

## 3.1. 费曼直觉

正式代码和实验代码的要求正好相反：正式代码要能编译、能过测试、能被别人读懂；实验代码是“写两行、跑一下、扔掉”。所以它们该待在不同地方，而实验区要同时满足三条：

1. **能直接用你正在写的包**（否则学数据库只能在纸上推）；
2. **跑起来只要一条命令**：`go run ./scratch/<名字>`；
3. **不会把正式代码的检查拖下水**：半成品实验不该让 `go build ./...` 变红。

社区里这类目录常见三种叫法：`scratch/`（一次性草稿）、`playground/`、`experiments/`。而 Go 官方文档《Organizing a Go module》讲的是 `cmd/`、`internal/`、`examples/` 这类**正式**布局 —— 其中 `examples/` 是给别人看的、稳定的示例，和“随手实验”不是一回事，别混。

## 3.2. 硬核机制

### 3.2.1. 落地的结构

```
db-go/
├── go.mod                  # module github.com/zz-cpp/db-go
├── AGENTS.md
├── README.md
├── cmd/dbshell/            # 正式可执行入口
├── internal/db/            # 正式实现（学习主体）
├── note/                   # 问答笔记
└── scratch/                # ← 新增：实验区，随时可以整个删掉
    ├── README.md           #   规则：命名 / 怎么跑 / 怎么隐身 / 怎么清
    ├── hello/main.go       #   最小模板：打印 Go 版本，确认链路是通的
    └── kvprobe/main.go     #   直接用 internal/db.KV 做实验（见下）
```

Listing 9: 一个实验 = 一个子目录 = 一个 package main

### 3.2.2. 三条硬规则（都是工具链定的，不是习惯）

| 规则 | 为什么 | 实测 |
| --- | --- | --- |
| 一目录只许一个 `func main()` | 同目录所有 `.go` 文件属于**同一个包**，两个 `main` 就是重复声明 | `scratch/hello/second.go:3:6: main redeclared in this block` |
| 目录名只影响导入路径 | `package main` 的包名固定，目录名只进包路径，连字符也收 | `go run ./scratch/slice-basics` 正常跑 |
| `_` / `.` 开头、`testdata`、`vendor` 会被 `./...` 跳过 | `go help packages` 明文规定 | `go list ./...` 里看不到 `_scratch/hi`， |

| | |
|---|---|
| | 但 `go run ./_scratch/hi` 照样跑 |

规则 3 的原文（`go help packages`）：

```
    File system patterns expanded with the "..." wildcard exclude the following:

    - Directories named "vendor"
    - Directories named "testdata"
    - Files and directories with names beginning with "_" or "."
    - Directories that contain a go.mod file
    - Directories matching an ignore directive in a module's go.mod file

    These can be included by either using them in the prefix, or changing into the
directories.
    For example, "./..." won't match a "./testdata/foo" package, but "./testdata/..." will.
```

Listing 10: 被 `./...` 跳过 ≠ 不能跑：显式写出路径依然可以 —— 所以「`_` 前缀」是把半成品隐身起来的正规做法

#### 3.2.3. 为什么实验区必须待在模块内

`internal/` 有一条导入规则：路径中含 `internal` 元素的包，只有**它父目录树之内**的包能 import。`github.com/zz-cpp/db-go/internal/db` 的父目录就是模块根，于是只有本模块内的包用得了它。

实测“家目录另开一个模块 + `replace` 指过来”的方案：

```
    // /tmp/sbout: go.mod 里 replace 指向本仓库，main.go 里 import internal/db
    $ go run .
    package sbout
      main.go:6:2: use of internal package github.com/zz-cpp/db-go/internal/db not allowed
```

Listing 11: 模块边界 + internal 规则联手拦下

> 📌 **核心要点** 选位置的判据不是“哪个路径好看”，而是**你要不要用到本项目的包**：要，实验区就必须在 `github.com/zz-cpp/db-go` 模块内（`scratch/`）；不要（纯语言实验），放哪都行。

### 3.3. 陷阱与易错

1. **“一个实验一个文件”是错的**：同目录两个 `main.go` 直接 `main redeclared in this block`，必须一个实验一个**目录**。
2. **`go run` 的参数是包路径，不是文件名**：`go run ./scratch/hello` 会编译该目录下所有 `.go`；而 `go run ./scratch/hello/main.go` 走的是**单文件模式**，同目录的其它文件根本不参与编译（实测：即使目录里躺着另一个 `main`，它也能跑）。两种模式混着用很容易看不懂报错。

3. **实验也可以写成测试**：想验证一串语义时，与其打一堆 `fmt.Println`，不如在同目录写 `xxx_test.go`（`package main`），然后 `go test ./scratch/<名字>`。
4. **`scratch/` 会被 `go build ./...` 一起检查**：这既是优点（顺手查笔误）也是负担（半成品会让它变红）。真想让某个实验隐身，把它挪进 `_scratch/`。
5. **`kvprobe` 现在跑不了不是它自己的错**：它 import 的 `internal/db` 卡在 `missing function body`（`Del` 没实现）。这恰好把待办变成了可见的红灯。

## 3.4. 关键点

1. 一个实验 = 一个子目录 = 一个 `package main`；跑法 `go run ./scratch/<名字>`。
2. 三条硬规则：同目录只能一个 `main()`；目录名只影响导入路径；`_`、`.`、`testdata`、`vendor`、含 `go.mod` 的目录会被 `./...` 跳过（出处 `go help packages`）。
3. 位置由 `internal` 规则决定：要用本项目的包，实验区就必须在本模块内。
4. `examples/`（给别人看的稳定示例）≠ `scratch/`（自己的一次性实验）。

## 3.5. 变式检验

1. 想让某个实验不影响 `go build ./...`，怎么办？→ 目录名前加 `_`（或放进 `testdata/`）；显式路径仍然能跑。
2. 实验目录取名有讲究吗？→ 对 `package main` 没有（连字符都行）；但要被别人 import 的库目录，名字就得是合法标识符风格。
3. 直接把实验写在 `internal/db/` 里行不行？→ 能编译，但会污染正式实现，而且包里的每个文件都被算进构建；实验要扔时容易误删。
4. `go run ./_scratch/x` 跑一个 `./...` 看不见的目录，行吗？→ 行，实测通过：隐身只针对通配符匹配，不针对显式路径。

## 3.6. 同类归类

| 目录 | 被 `./...` 匹配 | 用途 |
|---|---|---|
| `cmd/dbshell/` | ✓ | 正式可执行入口（对外发布） |
| `internal/db/` | ✓ | 正式实现，模块外导不进来 |
| `scratch/` | ✓ | 自己的一次性实验（本次新增） |
| `_scratch/` 或 `testdata/` | ✗ | 想让半成品隐身时的去处 |
| `examples/` | ✓ | 给别人看的稳定示例（本仓库还没有） |

## 3.7. Go 语言机制补充

**`./...` 到底怎么展开**：`...` 匹配“零个或多个路径元素”，展开时**排除** `vendor`、`testdata`、`_` / `.` 开头的文件与目录、以及含 `go.mod` 的目录（子模块边界）。这套规则是 Go 命令自身的行为（见 `go help packages`），不是 shell 通配 —— shell 那一层根本没参与。

**`internal` 的可见性**：这是编译器的导入检查，规则一句话 —— 导入路径中**最后一个** `internal` 之前的那段前缀（这里是 `github.com/zz-cpp/db-go`）决定了谁有资格导入它。它让“对外 API”和“内部实现”用目录结构区分开，不必引入额外的构建标签或可见性关键字。

**模块边界**：含 `go.mod` 的目录会被 `./...` 当成另一个模块、不再向下展开 —— 将来真要在仓库里放独立模块（比如 `tools/go.mod`），两边不会互相干扰；想访问它得 `cd` 进去。

**验证命令**：

```
$ go run ./scratch/hello      # 环境通了；Go go1.27.1-X:nodwarf5
$ go list ./...               # 四个包：cmd/dbshell、internal/db、scratch/hello、scratch/kvprobe
$ go run ./scratch/kvprobe    # 现在报 internal/db/kv.go:28:6: missing function body（待办）
```

Listing 12: 都在本仓库实测过；`go list ./...` 是“谁被算进项目”最直接的证据

## 3.8. 本次操作与当前状态

### 新增的文件

1. `scratch/README.md` —— 实验区规则（怎么加、怎么跑、三条硬规则、隐身技巧、为什么必须在模块内、怎么清）；
2. `scratch/hello/main.go` —— 最小模板，`go run ./scratch/hello` 实测输出 `环境通了；Go go1.27.1-X:nodwarf5`；
3. `scratch/kvprobe/main.go` —— 直接用 `internal/db.KV` 演练 `Open/Set/Get/Del`，**现在编译不过**（`Del` 缺函数体），补完即绿。

### 四条实测（在 /tmp 的对照沙盒里做，未进仓库）

```
$ go list ./...               # 只列出 sbtest/scratch/hello：_scratch/hi 与 testdata/td 被跳过
$ go run ./_scratch/hi        # 隐身目录用显式路径照样跑 ✓
$ go run ./scratch/hello      # 同目录两个 main：main redeclared in this block ✓
$ go run .                    # 模块外 + replace：use of internal package ... not allowed ✓
```

Listing 13: 四条实测支撑了上面的全部设计决策

**没有改动**：`internal/db/`、`cmd/`、`AGENTS.md` 都没动。`scratch/` 是新增目录，不想要就 `rm -rf scratch`。

# 4. gokw 怎么用

## 4.1. 费曼直觉

`gokw` 是前面那节里装的 **fish 函数**（不是 Go 程序）。它把你的终端变成一个“名字速查台”，回答三类问题：

1. **Go 到底有哪些关键字？** → `gokw`
2. **这个词能不能当变量名？** → `gokw <词>`
3. **规范原文怎么写的？** → `gokw spec`

## 4.2. 硬核机制

### 4.2.1. 四条命令

| 命令 | 做什么 | 输出要点 |
|---|---|---|
| `gokw` | 25 个关键字按用途分五组，每词一句中文说明 | 一屏看完，末尾给用法提示 |
| `gokw <词>` | 判定一个词属于哪一类名字 | 四种结论之一，见下表 |
| `gokw spec` | 打印离线语言规范 `#Keywords` 一节原文 | 5 行关键字表 + reserved 那句 + 在线链接 |
| `gokw -h` | 用法一览 | 4 行 |

### 4.2.2. `gokw <词>` 的四种结论

| 输入 | 结论 | 含义与实测 |
|---|---|---|
| `gokw for` | 关键字 | 保留字，任何地方都不能当标识符；输出还会给一句它干什么 |
| `gokw len` | 预声明标识符（builtin） | 语言自带的类型/常量/函数；**可以**被你的同名声明遮蔽 |
| `gokw myvar` | 都不是 | 可以随便当变量名/函数名 |
| `gokw Range` | 都不是 + **导出提示** | 首字母大写 = 包外可见；那是可见性，不是保留字 |
| `gokw _` | 空白标识符特例 | 能出现在赋值左侧却从不绑定，也不能当值用 |

### 4.2.3. 为什么它不会过期

清单**不写死**：每次运行都用一条 awk 现抽 `$GOROOT/src/go/token/token.go` 里 `keyword_beg` … `keyword_end` 之间的词：

```
    $ awk '/^\tkeyword_beg/{f=1; next} /^\tkeyword_end/{f=0} f && $1 ~ /^[A-Z]+$/ {print tolower($1)}' \
          "$(go env GOROOT)/src/go/token/token.go"
```

Listing 14: 抽出来的 25 个词再跟函数里写死的“分组 + 中文说明”表对照，对不上就红字报警：Go 升级后关键字有增减，它会自己告诉你

### 4.2.4. fish 是怎么找到它的

```
$ type gokw
# gokw 是一个函数
# 通过 source 定义，来自 ~/.config/fish/functions/gokw.fish

$ functions -v gokw   # -v 连注释一起打印，等于看源码
$ funced gokw         # 在编辑器里改它（改完 fish 会自动 source）
```

Listing 15: fish 的“自动加载”（autoload）：只有你第一次敲 gokw 时，才去 functions/ 目录找同名文件并 source —— 所以不需要往 config.fish 里加任何东西

## 4.3. 陷阱与易错

1. **`gokw` 只判名字，不判语法**。它回答的是“这个词能不能当标识符”，不是“你这行代码对不对”。想知道 `for` 怎么写，用 `gokw spec` 拿到规范原文，再去对应章节看语法产生式。
2. **大小写敏感**：`gokw Range` 说“不是关键字”是对的 —— Go 区分大小写，`Range` 只是个首字母大写的普通标识符（导出名）。真正被保留的只有全小写的 `range`。
3. **`_` 不是普通名字**：它是空白标识符（blank identifier）。`_ = f()` 合法（算出来就扔）、`x := _` 编译不过（`cannot use _ as value`）、而且它能在一行里重复出现：`_, _, err := f()`。前三条结论表里它是单独一行，就是因为“都不是”这个结论对它具有误导性。
4. **它依赖一个完整的 Go 发行版**：需要 `go` 在 `PATH` 里、且 `$GOROOT/src/go/token/token.go` 与 `$GOROOT/doc/go_spec.html` 存在（都是发行版自带的）。找不到文件会打印一行错误并以状态 1 退出，不会静默出错。
5. **写它时踩到的三个 fish 坑**（今天都修了，值得记）：

```
① fish 4.9 的 string match 默认 glob 已不支持 [A-Z] 字符类 ——
   string match -q '[A-Z]*' -- Range   # 退出码 1（判成「不匹配」）
   string match -qr '^[A-Z]' -- Range  # ✓ 改用 -r 正则

② test "$w" = -h 会把 -h 当选项 → 用 contains -- $w -h --help

③ fish 的 grep 被包装成 rg（function grep --wraps=rg）→
   脚本里写 command grep 才能拿到 GNU grep 的语义
```

Listing 16: 前两条直接影响了判定结果：第 ① 条曾让“大写导出提示”静默失效

> ⚠ **陷阱预警** `gokw` 是你自己的函数，不是 Go 官方工具：它的**说明文字**（每词那句中文）是我写的，可能有偏差；但**清单和判定依据**都来自本机 Go 发行版（`token.go` 与 `go doc builtin`），那部分永远和你的 Go 版本一致。要权威表述，看 `gokw spec` 或查 `go/token`。

## 4.4. 关键点

1. 四条命令：`gokw`（全表）、`gokw <词>`（判定）、`gokw spec`（规范原文）、`gokw -h`（用法）。
2. 判定有四类结论：关键字 / 预声明标识符 / 都不是 / 空白标识符特例。

3. 关键区分：**关键字不可遮蔽，内建名可遮蔽** —— 这正是 `len := 42` 能编译而 `for := 42` 不行的原因。
4. 文件在 `~/.config/fish/functions/gokw.fish`；删掉它就完全消失（没改过你任何其它文件）。
5. 清单每次现抽，所以它不会随 Go 升级而过期；对不上会红字报警。

## 4.5. 变式检验

1. 我想加一个自己的词条说明？→ 改 `switch $w` 里对应的 `case`，或者直接在 `functions -v gokw` 里看结构。
2. 换一台机器怎么带走？→ 复制 `~/.config/fish/functions/gokw.fish` 过去即可（Go 发行版自带 `token.go` 与 `go_spec.html`，所以到哪都能现抽）。
3. 想看某个关键字的完整语法？→ `gokw spec` 拿到规范位置，再在 `go_spec.html` 里搜对应小节（如 `For_statements`）。
4. 不想要了？→ `rm ~/.config/fish/functions/gokw.fish`；想同时保留查看记录可以 `funcsave gokw` 到别处备份。

## 4.6. 同类归类

三个工具分工不同，别混：

| 工具 | 查什么 | 底层 |
|---|---|---|
| `gokw` | 关键字 / 名字能不能用 | `go/token` 源码 + `go doc builtin` + 离线规范 |
| `godocbat <包>` | 包与 API 的用法、方法清单 | `go doc`（+ bat 高亮） |
| `godocsrc <符号>` | 某个 API 的实现 | `go doc -src` |
| `gokw spec` | 语言的规范性描述 | `$GOROOT/doc/go_spec.html` |

## 4.7. Go 语言机制补充

**`go env GOROOT` 而不是硬编码 `/usr/lib/go`**：`GOROOT` 是“Go 发行版装在哪儿”。`go env GOROOT` 让 go 命令自己回答，于是换机器、换安装方式（发行版包 / 官方 tarball / gvm）都不用改脚本。你的 Go 装在 `/usr/lib/go`，但函数里不写这个路径。

**`go doc builtin` 的输出有缩进陷阱**：里面 `type` 条目顶格，而 `func` 条目带 4 个空格缩进（`    func len(v Type) int`），所以判定正则必须写成 `^[[:space:]]*(func|type|const|var) <词>\b` —— 漏掉 `^[[:space:]]*` 就会把 `len`、`nil`、`append` 全判成“都不是”。

**空白标识符在源码里的位置**：`go/token.IsIdentifier("_")` 返回 **true**（它由下划线组成、非空、首字符非数字），所以「`_` 不是关键字」这句严格来说是对的 —— 但它在语法上有专门规则（永远不绑定），这就是函数里单独给它开一个分支的原因。

**验证命令**：

```
$ fish -c 'gokw'        # 25 个关键字分组表 ✓
$ fish -c 'gokw for'    # 关键字 ✓
$ fish -c 'gokw len'    # 预声明标识符 ✓
$ fish -c 'gokw myvar'  # 都不是 ✓
$ fish -c 'gokw Range'  # 都不是 + 导出提示 ✓
$ fish -c 'gokw _'      # 空白标识符特例 ✓
$ fish -c 'gokw spec'   # 离线规范 255-266 行原文 ✓
$ fish -c 'gokw -h'     # 用法 ✓
```

Listing 17: 八条全部实测通过（用 fish -c 跑，等价于交互式敲命令）

## 4.8. 本次操作与当前状态

**本次改了 `~/.config/fish/functions/gokw.fish` 三处**

1. 新增 `_` 特例分支：以前 `gokw _` 会说“可以随便当变量名”，那是误导 —— 现在讲清空白标识符的三条规则；
2. “都不是”分支新增首字母大写提示：`gokw Range` 现在会补一句“这是导出名，不是保留字”；
3. 修掉一个静默失效的 bug：`string match -q '[A-Z]*'` 在 fish 4.9 里判不出大写（glob 不再支持字符类），改用 `string match -qr '^[A-Z]'`。

**实测**：上表八条命令逐条跑过，输出与预期一致。

**没有改动**：Go 项目本体、笔记以外的文件都没动；`gokw` 仍然只占一个文件，删掉即消失。

# Index of Listings


Listing 1 它把 range 当成“当前包里一个叫 range 的符号”，找不到就报这个 —— 而不是告诉你「这是关键字」 . . . . . 6
Listing 2 `keyword_beg` / `keyword_end` 是一对哨兵：夹在中间的就是关键字 . . . . . 6
Listing 3 一次哈希查询就把“标识符还是关键字”分好类；没命中就是普通标识符 IDENT . . . . . 7
Listing 4 单字母标识符（i、x）不可能撞上关键字，所以干脆跳过查询 —— 热路径上的小优化 . . . . . 7
Listing 5 实测输出 42，编译通过 —— 换成任何关键字都不行 . . . . . 8
Listing 6 注意第二段里那句 unexpected keyword if —— 编译器明确告诉你它认出了关键字 . . . . . 8
Listing 7 最后一条就是 gokw 内部用的抽取命令，可以直接粘来核对当前 Go 版本 . 10
Listing 8 gokw 子命令与两个反例程序均实测（反例程序放在 /tmp/kwtest，未进仓库）；PDF 由 typst compile 零错误生成 . . . . . 10
Listing 9 一个实验 = 一个子目录 = 一个 package main . . . . . 11
Listing 10 被 `./...` 跳过 ≠ 不能跑：显式写出路径依然可以 —— 所以「`_` 前缀」是把半成品隐身起来的正规做法 . . . . . 12
Listing 11 模块边界 + internal 规则联手拦下 . . . . . 12
Listing 12 都在本仓库实测过；`go list ./...` 是“谁被算进项目”最直接的证据 . . . . . 14
Listing 13 四条实测支撑了上面的全部设计决策 . . . . . 14
Listing 14 抽出来的 25 个词再跟函数里写死的“分组 + 中文说明”表对照，对不上就红字报警：Go 升级后关键字有增减，它会自己告诉你 . . . . . 15
Listing 15 fish 的“自动加载”（autoload）：只有你第一次敲 gokw 时，才去 functions/ 目录找同名文件并 source —— 所以不需要往 config.fish 里加任何东西 . . . . . 16
Listing 16 前两条直接影响了判定结果：第 ① 条曾让“大写导出提示”静默失效 . . . . . 16
Listing 17 八条全部实测通过（用 fish -c 跑，等价于交互式敲命令） . . . . . 18